pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Covenants 1: The Anatomy of an Indenture

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the anatomy of an indenture

One document

Indenture	Boyd Gaming, 4.750% notes, 2031
Filed	8-K Exhibit 4.1, 2021-06-08
Saved	responses recorded 2026-10-08

Today

1. Parties, articles, Article 4
2. Section 1.01: the dictionary
3. Key covenants, read by regex
4. A schema: five pydantic models
5. First extraction, three checks
6. Prompt changes; five covenants
7. Guardrail, Excel, Git, AI check

The model returns a table. Code that does not involve the model decides which rows stay.

The rules for real documents

Every prompt asks what the indenture says: terms as JSON, a quote beside every number

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

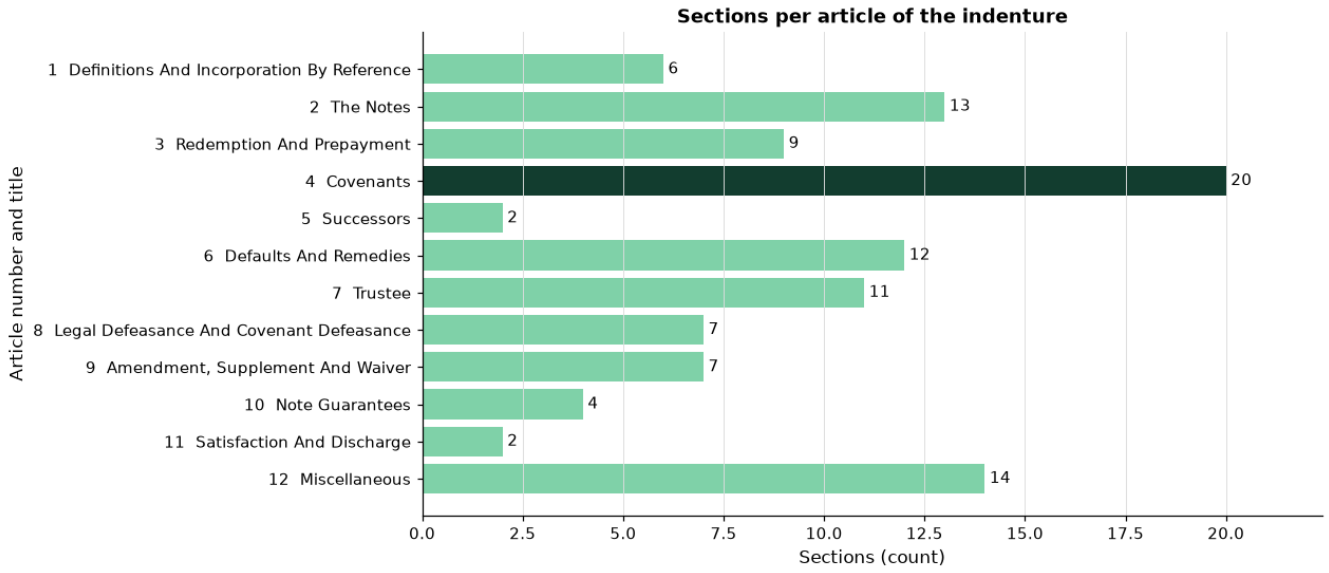
The system prompt goes first in every request and refuses an opinion, a grade, a comparison of issuers, or a view about the future.

What an indenture is

Who signs it

Company	the issuer
Guarantors	subsidiaries
Trustee	acts for the holders

Holders buy the notes; they do not sign.



Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

Story: the Trust Indenture Act of 1939

15 U.S.C. 77aaa and 77bbb(a), read 8 October 2026

77aaa, in full: This subchapter may be cited as the "Trust Indenture Act of 1939."

Investors are "adversely affected ... (1) when the obligor fails to provide a trustee to protect and enforce the rights and to represent the interests of such investors"

Section 12.01: "This Indenture is not and will not be qualified under, and does not and will not incorporate or include any of the provisions of, the TIA."

Sources: law.cornell.edu/uscode/text/15/77aaa; law.cornell.edu/uscode/text/15/77bbb

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 12.01.

Defined terms are the dictionary

148

definitions_dict: every quoted term

136

day 1's defined_terms

36 / 65

terms 4.12 uses, depth 1 / 2

The 12 day 1 missed: words between the term and the verb

Additional Lease
Change of Control
Consolidated Total Assets
Corporate Trust Office of the Trustee
Disqualified Stock
Investment
Net Cash Proceeds
Net Proceeds
Outstanding
Rating Decline
Responsible Officer
Subsidiary

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 1.01.

The key covenants, read by regex

at least 2.0 to 1.0

Consolidated Fixed Charge Coverage Ratio, 4.12(a)

101%

repurchase price after a Change of Control, 4.10

```
Permitted Liens (Section 1.01): [('$430,000,000', '7.5%')]
Permitted Investment (Section 1.01): [('$855,000,000', '15.0%'), ('$570,000,000', '10.0%')]
```

Section 4.14 writes no amount: its general basket is in the definition of Permitted Liens, Section 1.01.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 1.01.

How a schema works

The problem	A model returns text; a program needs fields
The schema	Five small classes list the fields and their types
Two uses	pydantic validates the reply and writes the schema into the prompt
No repair	A reply that does not fit fails, naming the field
What comes out	A Covenant object or an error

A schema: five pydantic models

```
class Quote(BaseModel):       section: str; text: str
class RatioTest(BaseModel):   name: str; threshold: float; unit: str; direction: str; quote: Quote
class Basket(BaseModel):      name: str; fixed_amount_usd: int | None; percent: float | None
                              percent_of: str | None; greater_of: bool; quote: Quote
class Covenant(BaseModel):    kind: Literal[...]; section: str; title: str
                              ratio_test: RatioTest | None; baskets: list[Basket]; quote: Quote
class CovenantPackage(BaseModel):  indenture: str; covenants: list[Covenant]
```

Every number carries a quote, and every quote carries a section: that is what makes a response checkable.

How the three checks work

The problem	A tidy number may not be in the document
Check 1	The section the value cites is in the table of sections
Check 2	Its quote is inside that section, after normalize_text
Check 3	A written form of the number is inside the quote
What comes out	One row per value: three true or false columns and passed

Three checks on every number

section_exists	the quote's section is in find_sections' table
quote_found	the quote is inside that section, after normalize_text
number_found	one of number_forms(value, unit) is in the quote, as a whole number

6 of 7

passed, the first prompt, saved response

41,147

characters in the user message

Credit Facilities: percent = 4.5 (%), cited as Section 4.12; failed: number_found

The text writes "4.5 times Consolidated EBITDA": a multiple, and the schema has no field for one.

How prompt tuning works

The problem	Which sentence of the prompt made the table better?
One change	Add one sentence, then read the same three checks
This text only	Never on the document that will test the prompt
What you control	The sentences and their order
What comes out	Values checked and passed per prompt, and a log to diff

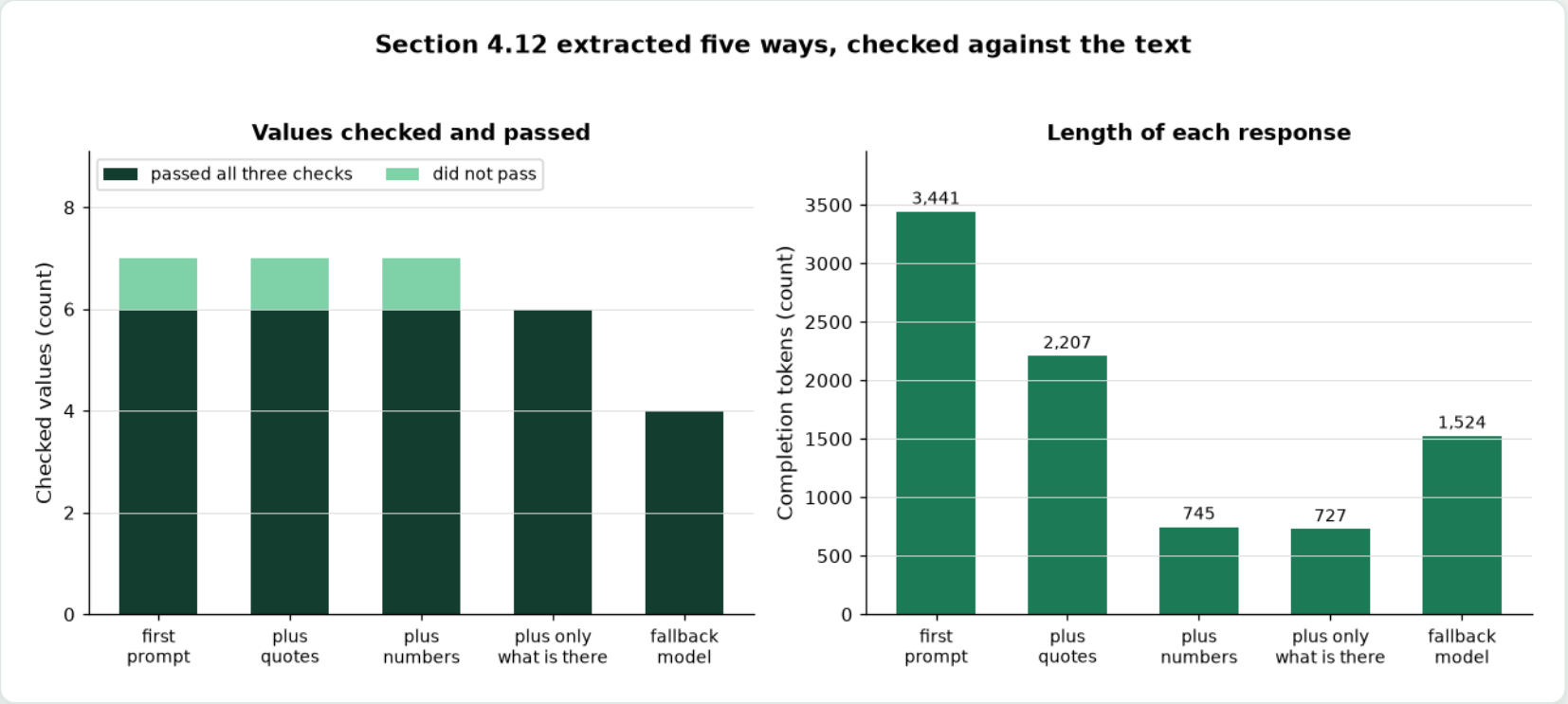
Prompt changes, one at a time

Prompt	Sentence added	Passed of checked
first	the schema only	6 of 7
change 1	quotes exact, one clause; cite 4.12 or 1.01	6 of 7
change 2	numbers as written; ratio with unit x	6 of 7
change 3	only baskets with an amount; null; NOT IN TEXT	6 of 6
fallback	the change 3 prompt, paid model, US$0.0015171	4 of 4

```
-boyd_2021,indebtedness,Credit Facilities: percent,4.5,%,4.12,True,False,nvidia/nemotron-3-super-120b-a12b:free,38876897
git diff -U0 extractions.csv: the 4.5 row in the first log, gone from the second
```

Chosen on this indenture only; the capstone's indenture stays closed.

Section 4.12 extracted five ways



6 of 7

first prompt: passed of checked

6 of 6

last prompt: passed of checked

Each bar is one saved response on one section, not a model ranked on a sample.

Five covenants in one checked table

12

values checked, five covenants

10

passed all three checks

2

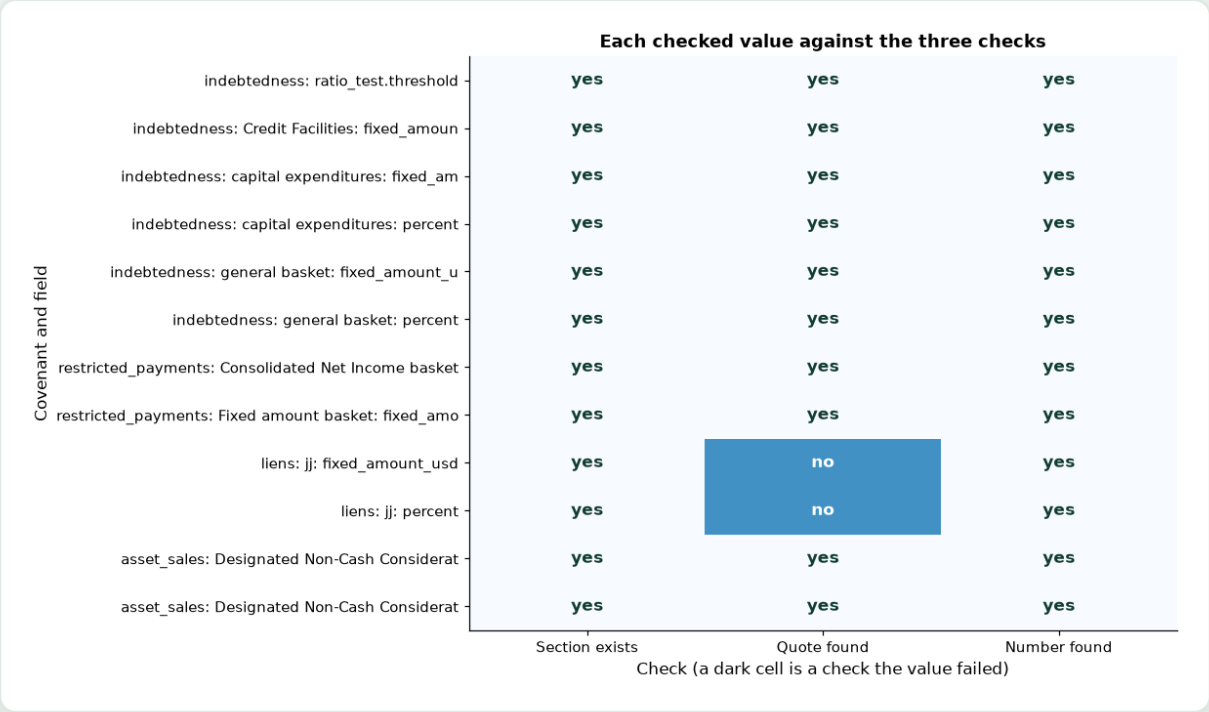
rejected, kept for a person

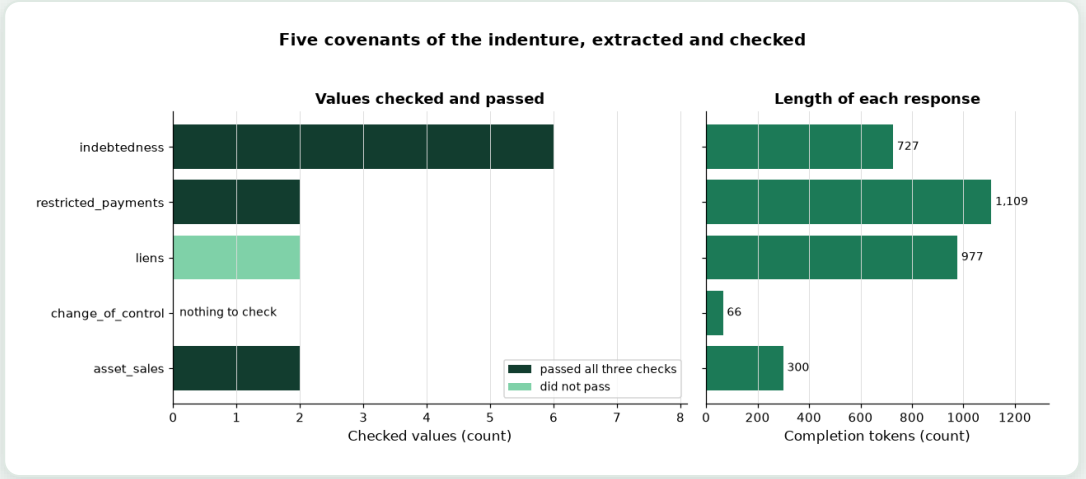
```
liens: jj: fixed_amount_usd = 430000000.0, Section 4.14: its quote is not in the section
liens: jj: percent = 7.5, Section 4.14: its quote is not in the section
the quote is in: Section 1.01
```

Section 4.10 returned no amount: its 101% is a price, and the schema has no field for it.

A rejected value is not a wrong value, and a confirmed value is not proven right.

Five covenants, drawn





12 values checked, 2 rejected: both in the liens rows, both on the quote check.

The guardrail

```
for text in sent + [textkit.SYSTEM_PROMPT]:
    textkit.assert_extraction_only(text)   # Course 4's word list, then OPINION_WORDS, whole words
```

10

instructions checked, every one passed

69

tests passed at the end of the day

Run it on the instruction, never on the document: an indenture says "will" on almost every page.

How the settings of an extraction request work

The problem	A reply that is cut off, or not JSON, cannot be parsed
response_format	Asks for one JSON object, so the reply can be parsed
max_tokens	Gives the reply room: 4,000 here against day 11's 2,000
The rest	Temperature 0, seed 42, reasoning off on the free model
What comes out	A response whose finish_reason you read before you parse

The forms of response_format

text (course sends it: no)

Plain text, the default.

json_schema (course sends it: no)

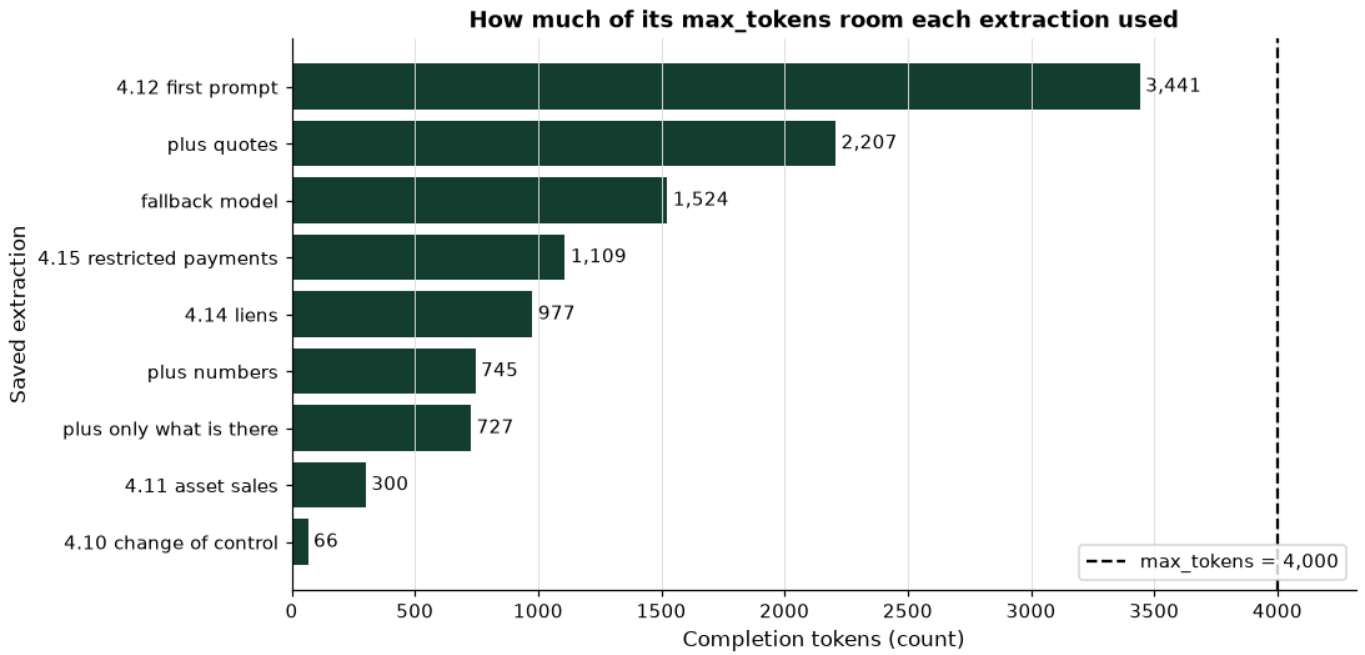
A JSON object that follows a schema sent with the request. The provider must support it.

json_object (course sends it: yes, in every extraction)

One JSON object. The instruction must still say which fields to write: the course's prompt carries the schema.

json_object guarantees JSON, not the fields: the instruction still carries the schema, and the checks decide every value.

How much room max_tokens gave



3,441 / 4,000

4.12 first prompt: the longest response, 86 of its room

All nine stopped on their own. A response at the dashed line stops with finish_reason length and is refused before parsing.

The same temperature, seed, max_tokens and response_format in every request; only extra_body differs.

Excel: the checked table

Excel side by side (tested in Excel)

`=ISNUMBER(SEARCH(B2,C2))  on the six saved quotes`	FALSE on all six (274 to 708 characters)
`=LEN(C2)<>LEN(SUBSTITUTE(C2,B2,""))  the same quotes`	TRUE on all six
`SEARCH with B = 4.5 times*EBITDA`	TRUE (a wildcard)
`SEARCH with B = CONSOLIDATED TOTAL ASSETS; FIND`	TRUE; FALSE
`=SEARCH(B2,C2)  a 300-character quote`	#VALUE! (FIND: TRUE)
`=ISNUMBER(SEARCH(TEXT(D2,"#,##0"),B2))  D2 = 285`	TRUE (Python: False)

For real quotes use the SUBSTITUTE form: literal, case-sensitive, the twin of Python's in.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the checker overwrites a failed value with the quote's own number and marks it passed.

Code that runs is not the same as code that is right.

What you can do now

- Take an indenture apart; build Section 1.01's dictionary
- Write a schema; read finish_reason, then parse_json
- Check every number: section, quote, number
- Change a prompt one sentence at a time; diff the log
- Checked table in Excel: SUBSTITUTE, not SEARCH

Week 3, Day 3 Covenants 2: baskets, carve-outs, and two vintages